

1. El DOM y las API de navegador

1.1. Las API de navegador

Los navegadores ofrecen una serie de funcionalidades para interactuar con el código de las páginas web a través de JavaScript. Estas funcionalidades reciben el nombre de **API de navegador** (*browser APIs*) o **API de cliente** (*web APIs*).

! IMPORTANTE

No hay que confundir las API de cliente con las API de servidor.

Existe un gran número de API de navegador, y además están en continua evolución. Algunas de las más útiles son:

- **DOM.** Proporciona acceso a la **estructura del documento** (normalmente **HTML**), de manera que se pueda leer y modificar desde JavaScript.
- **Web Storage.** Proporciona herramientas para **almacenar información** en el navegador.
- **Geolocation.** Permite el acceso a la **localización** del usuario. Es muy útil, sobre todo cuando se carga la página desde un dispositivo móvil.
- **MediaStream.** Permite enviar y recibir **audio y vídeo** y trabajar con dispositivos como **cámaras web** o **micrófonos**.
- **Drag and Drop.** Proporciona la funcionalidad de **arrastrar y soltar** para interactuar con elementos y archivos en las páginas web.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, dispones de un enlace al listado completo de API de navegador.

El acceso a las API del navegador se realiza a través del **objeto global** del navegador. Este objeto global recibe el nombre de **window**.

! IMPORTANTE

En otros entornos (como, por ejemplo, **NodeJS**), **window** no está definido. En su lugar, se define el objeto global como **global**.

Recientemente se ha añadido al lenguaje el objeto **globalThis**, que se utiliza para hacer referencia al objeto global, independientemente del entorno en el que estemos. Así, en navegadores, **globalThis = window**, mientras que en NodeJS, **globalThis = global**.

! IMPORTANTE

Como su propio nombre indica, en principio las API de navegador solo están definidas en los navegadores. Aunque es posible que también estén disponibles en otros entornos mediante **librerías** que las emulen.

1.2. El DOM: introducción y navegación

En el módulo de *Lenguaje de Marcas y Sistemas de Gestión de Información* de primer curso se realizó una introducción al DOM, así que en esta sección realizaremos un recordatorio de los conceptos básicos y profundizaremos en los aspectos más avanzados.

DOM significa **modelo de objeto de documento**, o *document object model*. Proporciona un acceso a la estructura del documento en forma de árbol jerárquico.

! IMPORTANTE

El DOM también proporciona acceso a otros formatos de documento, como el estándar **SVG** de gráficos vectoriales o **XML**. En esta unidad, nos centraremos en HTML exclusivamente.

! IMPORTANTE

La estructura del DOM de una página web puede verse a través de la pestaña *Elementos* o *Inspector* de las herramientas de desarrollo.

La estructura de árbol del DOM define una serie de **nodos**, cada uno de los cuales representa una parte del documento. Por ejemplo, las **etiquetas HTML** se convierten en nodos de tipo **elemento**, mientras que los textos se convierten en nodos de tipo **texto**. Cada uno de esos nodos proporciona acceso a unos **objetos**, que definen **propiedades** y **métodos** con los que interactuar.

! IMPORTANTE

Existen 12 diferentes tipos de nodos en HTML. En este apartado nos centraremos en los nodos de tipo **elemento**, los nodos de tipo **texto** y los nodos de tipo **atributo**. Otros tipos de nodo de interés son los nodos de tipo **comentario**, el nodo de tipo **documento** y los nodos de tipo **DocumentFragment**, que se utilizan en las plantillas HTML.

Dado que se trata de una estructura jerárquica, al referirnos a las **relaciones entre nodos** podemos hablar de **padres**, **hijos**, **hermanos**, **descendientes** y **ancestros**.

Para acceder a los nodos del documento, existen las siguientes propiedades:

- **document** (o **window.document**). Es el punto de entrada al DOM. Hace referencia al **documento** [archivo HTML].
- **document.documentElement**. Es el elemento raíz, es decir, la etiqueta **<html>** del documento.
- **document.head**. Hace referencia a la etiqueta **<head>** del documento.
- **document.body**. Hace referencia a la etiqueta **<body>** del documento.

Una vez hemos accedido a un nodo del DOM, podemos **navegar** a través de sus descendientes o ancestros mediante las siguientes propiedades:

- **parentNode**, **childNodes**, **firstChild**, **lastChild**, **previousSibling**, **nextSibling**. Hacen referencia al nodo padre, hijos (colección), primer hijo, último hijo, anterior hermano y siguiente hermano, respectivamente.
- **parentElement**, **children**, **firstElementChild**, **lastElementChild**, **previousElementSibling**, **nextElementSibling**. Hacen referencia a los **nodos de tipo elemento** (etiquetas HTML) padre, hijos (colección), primer hijo, último hijo, anterior hermano y siguiente hermano, respectivamente.

! IMPORTANTE

En la lista anterior es importante tener en cuenta que si se incluye la palabra **Element** se hace referencia a nodos de **tipo elemento**, mientras que, si no, se hace referencia a cualquier tipo de nodo.

! IMPORTANTE

Tanto `childNodes` como `children` devuelven una **colección** de nodos, a diferencia del resto de propiedades, que devuelven únicamente un nodo. Las colecciones de nodos tienen las siguientes **características**:

- No son *arrays*, por lo que no tienen acceso a las funciones de *arrays* de JavaScript (por ejemplo, no se puede ejecutar `map`, `filter` o `reduce` sobre ellas).
- A pesar de ello, sus elementos son accesibles a través de **índices** mediante el operador `[]`.
- Son **iterables**, por lo que se pueden iterar mediante `for...of`.

La mayoría de las veces, no es interesante acceder al DOM a partir del elemento raíz y navegar a través de sus descendientes, sino que nos interesa **realizar una búsqueda** para localizar un nodo o una colección de nodos que cumplan una condición. Para ello, podemos utilizar las siguientes funciones:

- `document.getElementById` o `querySelector`. Devuelven un **elemento**. En el primer caso, se debe indicar el **id** del elemento; en el segundo, se puede indicar un **selector CSS** para la búsqueda, y devuelve el primer elemento que coincida con esta búsqueda.
- `getElementsByTagName`, `getElementsByClassName` o `querySelectorAll`. Devuelven una **colección** de elementos por nombre de etiqueta, nombre de clase o selector CSS, respectivamente.

En el siguiente ejemplo puedes ver el funcionamiento de algunas de las funciones que hemos visto hasta ahora:

```
// 'getElementById' solo puede ejecutarse sobre 'document'
// La variable 'principal' apunta al elemento con 'id = principal'
let principal = document.getElementById("principal");

// Obtenemos todos los párrafos del documento
let parrafos = document.getElementsByTagName("p");

// 'primerImportante' apunta al primer elemento de los que cumpla la
condición
// dada por el selector CSS
let primerImportante = document.querySelector("#principal p.importante");

// 'querySelectorAll' puede ejecutarse sobre 'document' o sobre otro
elemento
// Estas dos líneas realizan la misma búsqueda
let importantes1 = document.querySelectorAll("#principal p.importante"); //
Se busca en todo el documento
let importantes2 = principal.querySelectorAll("p.importante"); // Se busca
dentro de '#principal'

// 'querySelectorAll' devuelve una colección.
// Recorremos dicha colección
for (let importante of importantes1) {
    console.log(importante);
}
```


! IMPORTANTE

`document.getElementById` solo puede ejecutarse sobre **document**; el resto de funciones de búsqueda que hemos indicado pueden ejecutarse sobre **document** y sobre un elemento, en cuyo caso realizarán la búsqueda entre los nodos descendientes de dicho elemento.

Las siguientes funciones son también muy útiles para trabajar con nodos:

- `elem.matches(SELECTOR_CSS)`. Comprueba si un elemento cumple las condiciones dadas por el selector CSS.
- `element.closest(SELECTOR_CSS)`. Devuelve el ancestro más cercano que cumpla las condiciones dadas por el selector CSS.
- `element.contains(ELEMENT)`. Comprueba si un elemento es descendiente de otro.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A01_01.html`, con el código de ejemplo.

1.3. El DOM: modificación

La manera más sencilla de realizar modificaciones sobre el DOM es utilizar las propiedades `innerHTML`, `outerHTML` o `textContent`. Todas ellas reemplazan el contenido del nodo seleccionado. `textContent` solo crea texto, mientras que `innerHTML` y `outerHTML` crean código HTML. `innerHTML` reemplaza el contenido del elemento, mientras que `outerHTML` reemplaza también el elemento indicado.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A01_02.html`, con el código de ejemplo.

! IMPORTANTE

`textContent`, `innerHTML` y `outerHTML` no son funciones, sino que son propiedades que se deben asignar a un nuevo valor.

Es posible añadir contenido al ya existente, mediante, por ejemplo, `elemento.innerHTML += "NUEVO CONTENIDO"`; . Sin embargo, el HTML que hubiera antes volvería a procesarse (si hubiera imágenes, por ejemplo, desaparecerían y volverían a cargarse de nuevo).

También es posible crear nuevos nodos mediante:

- `document.createElement(nombre_etiqueta)`. Crea un nuevo elemento del tipo indicado. No hay que indicar `<` `>`.
- `elem.cloneNode()`. Devuelve una copia del nodo indicado. Si se llama a la función con el parámetro `true`, incluye todos sus descendientes.

En ambos casos, los nodos creados no se añaden automáticamente al documento. Para ello hay que utilizar alguno de los siguientes métodos:

- `append`. Añade un nodo al final.
- `prepend`. Añade un nodo al principio.
- `before`. Añade un nodo antes que otro.
- `after`. Añade un nodo después que otro.
- `remove`. Elimina un nodo.
- `replaceWith`. Reemplaza un nodo con otro.

 EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A01_03.html`, con el código de ejemplo.

1.4. Atributos y clases

A través del DOM es posible **acceder a los atributos** de los elementos HTML y **modificarlos**. Las funciones disponibles son:

- `elem.hasAttribute(nombre)`. Permite comprobar si existe un atributo.
- `elem.getAttribute(nombre)`. Lee el valor del atributo.
- `elem.setAttribute(nombre, valor)`. Modifica el valor del atributo.
- `elem.removeAttribute(nombre)`. Elimina un atributo.
- `elem.attributes`. Devuelve una colección de los atributos del elemento.

Además, los **nodos del DOM** tienen definidas unas **propiedades**. Para acceder a la propiedad del DOM de un elemento se utiliza la notación de objetos de JavaScript (por ejemplo, `campoFormulario.value`), mientras que para obtener el valor del atributo se usa `campoFormulario.getAttribute("value")`.

 IMPORTANTE

Algunas de estas propiedades se corresponden con los **atributos HTML** del elemento asociado, **pero no siempre es así**.

Algunas de las **propiedades del DOM** más utilizadas son:

- **hidden**. Si se establece a **true**, **oculta** el elemento de manera equivalente a **display: none** en CSS (es decir, el elemento no ocupa espacio en el flujo del documento).
- **value**. Almacena el **valor** del campo de formulario.
- **type**. Almacena el **tipo** del campo de formulario.

 IMPORTANTE

El atributo **value** y la propiedad **value** del DOM **no son equivalentes**:

- Si se **cambia** el valor del atributo, cambia **también** el valor de la propiedad del DOM.
- Si se **cambia** el valor de la propiedad, **no se actualiza** el valor del atributo.

De esta manera, se puede hacer un cambio del valor del campo a través de la propiedad via JavaScript y seguir teniendo acceso al valor original almacenado en el atributo.

- **href**. Almacena la **URL** de un enlace.

Además de los atributos estándar, HTML permite definir **atributos con nombres personalizados**. Normalmente se utilizan junto con los nombres de clases para poder **marcar elementos** y hacer **referencia** a ellos desde JavaScript. Especial interés tienen los atributos cuyo nombre comienza por **data-**: dichos atributos reciben el nombre de **atributos del DataSet**, y están accesibles a través de `elemento.dataset.NOMBRE_ATRIBUTO`. A continuación, puede verse un ejemplo de uso:

 EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A01_04.html`, con el código de ejemplo.

Los atributos del DataSet son muy utilizados para **enriquecer el HTML** con información adicional o **añadir funcionalidad de manera declarativa**, como veremos en el apartado de eventos.

Por último, JavaScript define una serie de métodos para trabajar con **clases CSS**:

- **elem.className**. Devuelve una **cadena de texto** con el listado de **clases** definidas en el elemento.
- **elem.classList**. Es un **objeto** que define los métodos **add**, **remove**, **toggle** o **contains** para añadir, eliminar, conmutar o comprobar clases en un elemento.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A01_05.html**, con el código de ejemplo.

1.5. La API Geolocation

La API de geolocalización está disponible a través del objeto **navigator.geolocation**. Si se utiliza en una página, el navegador pedirá permiso al usuario para acceder a los datos de ubicación del dispositivo.



IMPORTANTE

No es necesario que el dispositivo desde el que se carga la página disponga de GPS para usar la API de geolocalización. La información puede obtenerse a través de la red (aunque de manera menos precisa).

El objeto proporciona los siguientes **métodos**:

- **geolocation.getCurrentPosition()**. Se utiliza para obtener la **ubicación actual**.
- **geolocation.watchPosition()**. Se utiliza para ejecutar una función **cada vez que cambia la ubicación**.

Ambas funciones admiten **tres parámetros**:

- Una función **callback** de un **parámetro** que se ejecuta si la petición para obtener la localización **tiene éxito**. El parámetro de la función es un objeto de tipo **GeolocationPosition**.
- Una función **callback** de un **parámetro** que se ejecuta si la petición **provoca un error**. El parámetro de la función es un objeto de tipo **GeolocationPositionError**. Este parámetro es opcional.
- Un objeto de tipo **PositionOptions** con opciones. Este parámetro es opcional.

Esta API es una API asíncrona: el resultado no se obtiene inmediatamente después de realizar la llamada, sino que se obtiene cuando se ejecuta la función de callback. En la siguiente unidad estudiaremos en profundidad este tipo de funciones.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A01_06.html**, con el código de ejemplo.